

Decision Trees and Random Forests: Techniques and Implementation

Dataset Description

The Iris dataset, utilized in this project, is a renowned dataset introduced by R.A. Fisher in 1936. It contains measurements of 150 iris flowers, distributed evenly across three species: Iris-setosa[0-49], Iris-versicolor[50-99], and Iris-virginica[100-149]. Each entry includes the following attributes:

1. Sepal Length
2. Sepal Width
3. Petal Length
4. Petal Width
5. Class (Species)

The dataset was sourced from the [UCI Machine Learning Repository](#).

Libraries and Tools Used:

NumPy

- Efficiently handles numerical computations and array manipulations.

Pandas

- Facilitates reading, writing, and manipulation of data in tabular format.

Matplotlib

- A visualization library built on NumPy arrays.
- Provides various plots such as line, bar, scatter, and histogram.

Pyplot:

- Pyplot makes Matplotlib work like MATLAB.

Seaborn

- Built on top of Matplotlib, it enhances statistical plotting.
- Provides beautiful default styles and colour palettes to make plots attractive.

Scikit-learn

- Provides tools for machine learning, preprocessing, and model evaluation. Key modules used include:
 - **train_test_split** for dataset splitting.
 - **DecisionTreeClassifier** and **RandomForestClassifier** for classification tasks.
 - Metrics such as **accuracy_score**, **classification_report**, and **confusion_matrix**.
-

Definitions:

iloc () function:

- Stands for "integer location".
- Used for accessing and retrieving data from Pandas DataFrame objects using integer-based indexing.

Train-Test Split:

- A method to divide the dataset into training and testing subsets to evaluate model performance.

random_state=0 :

- With random_state=0, we get the same train and test sets across different executions.

DecisionTreeClassifier:

- A Decision Tree Classifier is a type of algorithm that uses a tree-like structure to classify instances based on their feature values.

RandomForestClassifier:

- A random forest is a meta estimator that fits a number of decision tree classifiers on various sub-samples of the dataset and uses averaging to improve the predictive accuracy and control over-fitting.

n_estimators:

- The number of trees in the forest.

classifier.fit():

- The learning algorithm / estimator instance is first fitted to the model, i.e., it must

learn from the model. This is done by passing our training set to the fit method.

classifier.predict():

- The predict() will perform a prediction for each test instance, and it usually accepts only a single input.

confusion_matrix():

- Summarizes prediction results in matrix form, showing correct and incorrect predictions per class.

seaborn.heatmap() / sns.heatmap():

- Plot rectangular data as a color-encoded matrix.

Parameters:

(i) data: Here, the data input is the 'result'.

(ii) annot: bool or rectangular dataset, optional.

If True, write the data value in each cell. If an array-like object has the same shape as data, then use this to annotate the heatmap instead of the data. Note that DataFrames will match on position, not index.

In this example, in each confusion matrix, correctly and incorrectly classified test sample numbers are shown.

(iii) fmt: string formatting code to use when adding annotations.

G or g (General Format) suffix in Python: It can be interpreted as the total number of digits before the decimal; otherwise, it represents the total number of zeros after the decimal point but before the significant digits.

In this example, `fmt = 'g'` is used.

(iv) xticklabels, yticklabels: Here, sequentially, three classes, 'Setosa', 'Versicolor', and 'Virginica' are marked.

classification_report: Build a text report showing the main classification metrics.

(i) Precision: $TP / (TP + FP)$

(ii) Recall / Sensitivity: $TP / (TP + FN)$

(iii) F1 Score = $(2 * Precision * Recall) / (Precision + Recall)$

True Positive (TP): A true positive is an outcome where the model *correctly* predicts the *positive* class.

True Negative (TN): A true negative is an outcome where the model *correctly* predicts the *negative* class.

False Positive (FP): A false positive is an outcome where the model *incorrectly* predicts the *positive* class.

False Negative (FN): A false negative is an outcome where the model *incorrectly* predicts the *negative* class.

Support: Support is a measure of the number of times an item set appears in a dataset. Here, the item set is a test data sample.

Macro Average: Average the precision, recall, and F1 scores across all classes to get the final macro-averaged precision, recall, and F1 scores, respectively. You may do the hand calculations and cross-check the output you get.

Weighted Average:

$$\text{Weighted Average of Precision} = \frac{[\text{Sum (Individual Class Precision * Individual Class Support)}]}{\text{Total Support.}}$$

In a similar way, you can do it for the rest of the recall and F1 score. You may do the hand calculations and cross-check the output you get.

accuracy_score: In Python, the `accuracy_score` function of the `sklearn.metrics` package calculates the accuracy score for a set of predicted labels against the true labels.

Accuracy: $(TP + TN) / (TP + TN + FP + FN)$

The IRIS data used in the example has a total of 150 data samples that comprise 50 from each class of IRIS Setosa, IRIS Virginica, and IRIS Versicolor.

During the training and test split, 70% was used for training and 30% for testing purposes. 30% of 150 is 45.

That is the reason the total support count is 45.

For instance, depending on the output, among the 45, there can be 16 Iris Setosa, 11 Iris Virginica, and 18 Iris Versicolor. The same number will reflect during the prediction evaluations in the confusion matrix.

Preprocessing Steps:

Data Loading:

```
path = "https://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.data"
headnames = ['sepal-length', 'sepal-width', 'petal-length', 'petal-width', 'Class']
data = pd.read_csv(path, names=headnames)
```

Feature Extraction:

```
X = data.iloc[:, :-1].values # Features
y = data.iloc[:, 4].values # Labels
```

Data Splitting:

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.30, random_state=0)
```

Implemented Algorithms:

Decision Tree Classifier

- A tree-based algorithm that splits data at decision nodes based on the feature that provides the highest information gain.

Implementation:

```
from sklearn.tree import DecisionTreeClassifier
classifier = DecisionTreeClassifier()
classifier.fit(X_train, y_train)
```

Evaluation:

```
y_pred = classifier.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
```

Visualization of Confusion Matrix:

```
from sklearn.metrics import confusion_matrix
result = confusion_matrix(y_test, y_pred)
sns.heatmap(result, annot=True, fmt='g',
```

```
        xticklabels=['Setosa', 'Versicolor', 'Virginica'],
        yticklabels=['Setosa', 'Versicolor', 'Virginica'])
plt.ylabel('Prediction')
plt.xlabel('Actual')
plt.title('Confusion Matrix')
plt.show()
```

Random Forest Classifier

- An ensemble learning method that uses multiple decision trees to make predictions and aggregates their results.

Implementation (10 Trees):

```
from sklearn.ensemble import RandomForestClassifier
classifier = RandomForestClassifier(n_estimators=10)
classifier.fit(X_train, y_train)
```

Evaluation:

```
y_pred = classifier.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
```

Visualization of Confusion Matrix:

```
from sklearn.metrics import confusion_matrix
result = confusion_matrix(y_test, y_pred)
sns.heatmap(result, annot=True, fmt='g',
            xticklabels=['Setosa', 'Versicolor', 'Virginica'],
            yticklabels=['Setosa', 'Versicolor', 'Virginica'])
plt.ylabel('Prediction')
plt.xlabel('Actual')
plt.title('Confusion Matrix')
plt.show()
```

Implementation (20 Trees):

```
classifier = RandomForestClassifier(n_estimators=20)
classifier.fit(X_train, y_train)
```

Evaluation:

```
y_pred = classifier.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
```

Model Evaluation

Confusion Matrix

- Provides a summary of prediction results by comparing actual labels with predicted labels.

Classification Report

- Includes precision, recall, F1 score, and support for each class.

```
from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred))
```